Distributed Job Queue and Worker Management System

1. Problem Understanding

In this project, we are building a Distributed Job Queue and Worker Management System. The idea is to avoid making one computer handle every task by itself. Instead, a client can send a job to our application controller, which places it in a queue and decides which available worker should handle it. The selected worker performs the task and sends the result back. The controller also keeps track of each job, so the client can check whether it is waiting, being processed, completed, or has failed.

We use Mininet to create a small virtual network with multiple hosts and an Open vSwitch switch. Ryu is used as our SDN controller. It communicates with the switch through OpenFlow and helps control how network traffic is forwarded. Through this setup, we aim to show how a job is submitted, assigned to a worker, executed, and returned to the client while the network is managed using SDN.

2. System Architecture

APPLICATION LAYER

Client

TCP socket: submit job / receive response

Application Controller

Job Queue — stores jobs and tracks status

Scheduler — selects an available worker

Worker 1 | Worker 2 — execute assigned jobs

Result returns through the application controller to the client

NETWORK LAYER

Mininet hosts (h1, h2, h3, h4) connected through Open vSwitch switch s1

Ryu SDN Controller ⇄ OpenFlow ⇄ Open vSwitch

Note: This diagram represents the intended overall architecture. Current tests demonstrate successful job processing, Mininet host connectivity, and an active Ryu-to-switch connection. Claim end-to-end remote-worker integration only after verifying that the controller and worker communicate through network sockets.

3. Main Components

- **Client:** Submits computational jobs, checks job status, retrieves results, and lists workers.
- **Application Controller:** Receives client requests, manages job processing, and coordinates task execution.
- **Job Queue:** Stores submitted jobs and tracks their status during execution.
- **Scheduler:** Selects an available worker based on recorded workload; the current scheduler selects the worker with the lowest load.

- **Workers:** Execute assigned computational tasks and return results.
- **Mininet:** Creates a virtual network with hosts, switches, and links for testing on one machine.
- **Open vSwitch:** Acts as the virtual network switch and forwards packets between connected hosts.
- **Ryu SDN Controller:** Communicates with Open vSwitch through OpenFlow and manages network forwarding behaviour.

4. Demonstration Flow

1. The client submits a sum job for the integers from 60 to 70.
2. The controller acknowledges the request and returns a unique job ID.
3. The job is tracked in the queue.
4. The scheduler selects worker-1.
5. The worker calculates the sum.
6. The client checks the job status and receives the completed result.

60 + 61 + 62 + ... + 70 = 715

Example result observed in the demo:

```
job_type: sum
status: COMPLETED
worker_id: worker-1
attempts: 1
result: 715
error: null
```

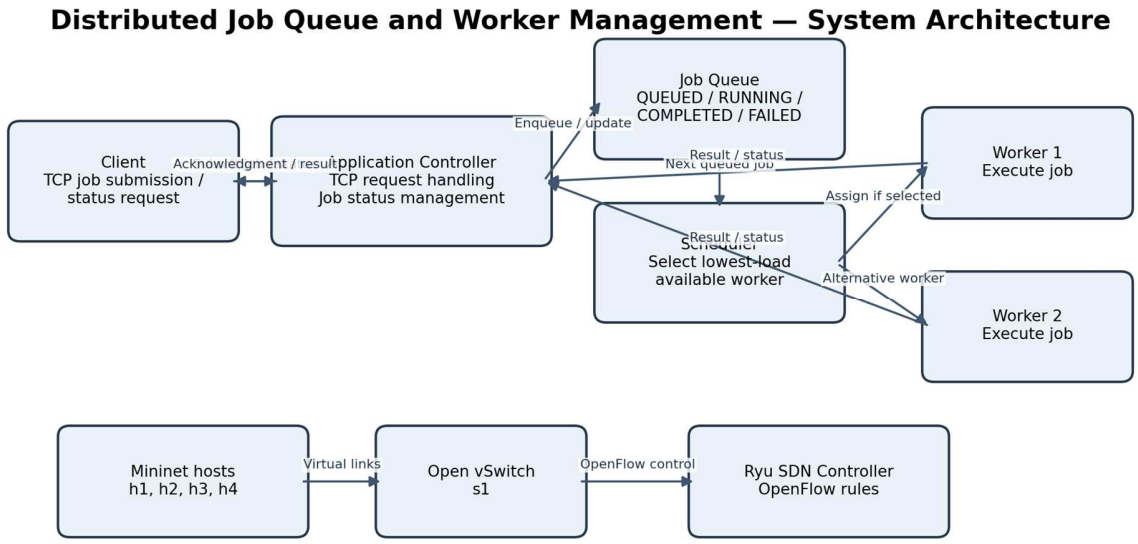
5. Evidence from Current Testing

- Application test: a sum job from 60 to 70 completed with result 715.
- Mininet connectivity: pingall reported 0% packet loss (12/12 received) across h1, h2, h3, and h4.
- SDN connection: Open vSwitch reported is_connected: true for the configured Ryu controller.
- Flow table: the switch displayed OpenFlow flow entries.

These results demonstrate application job processing and basic SDN network operation. They do not, by themselves, prove that a job was sent to a separate worker host over a network socket.

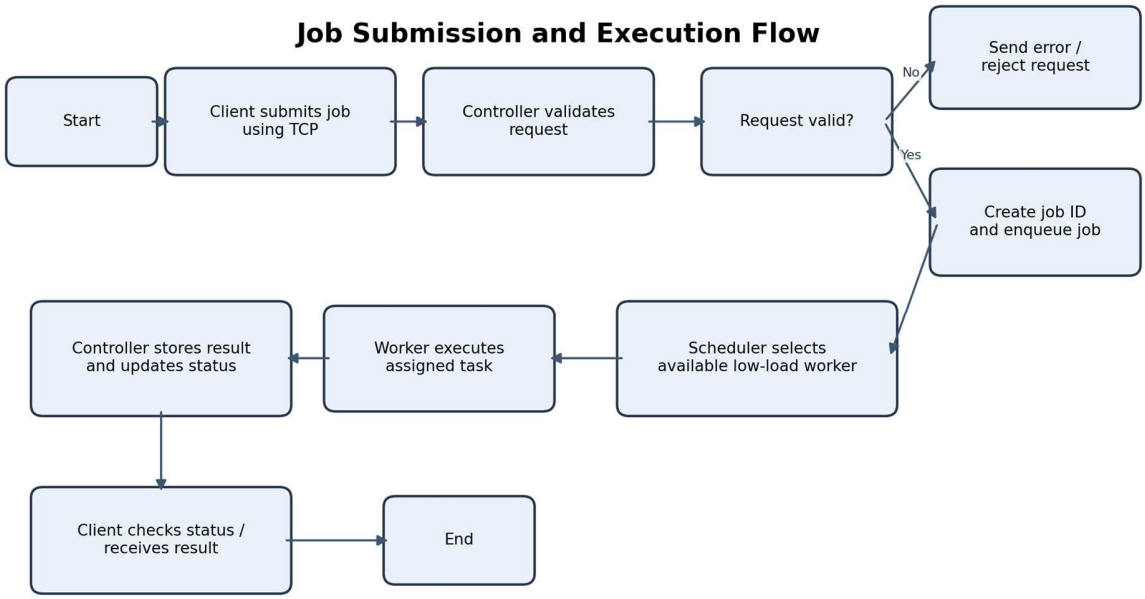
1 Overall System Architecture

The client sends its request to the application controller using a TCP connection. The controller then manages the queue, chooses a suitable worker, and sends the result back to the client. For the network part, we use Mininet hosts connected through an Open vSwitch. Ryu communicates with the switch using OpenFlow to manage packet forwarding.



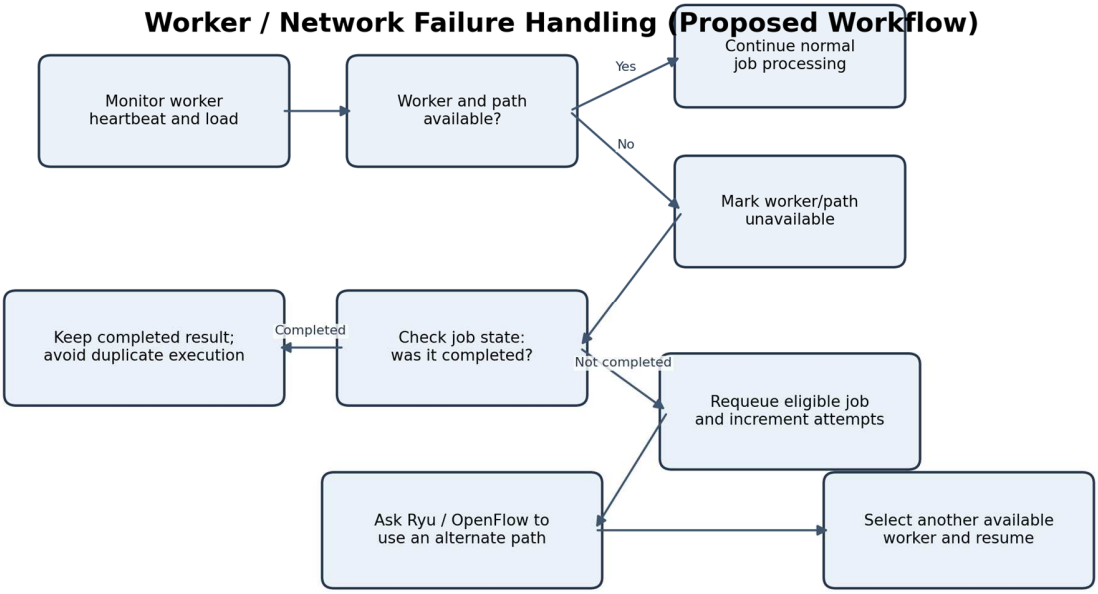
2 Job Submission and Execution Flowchart

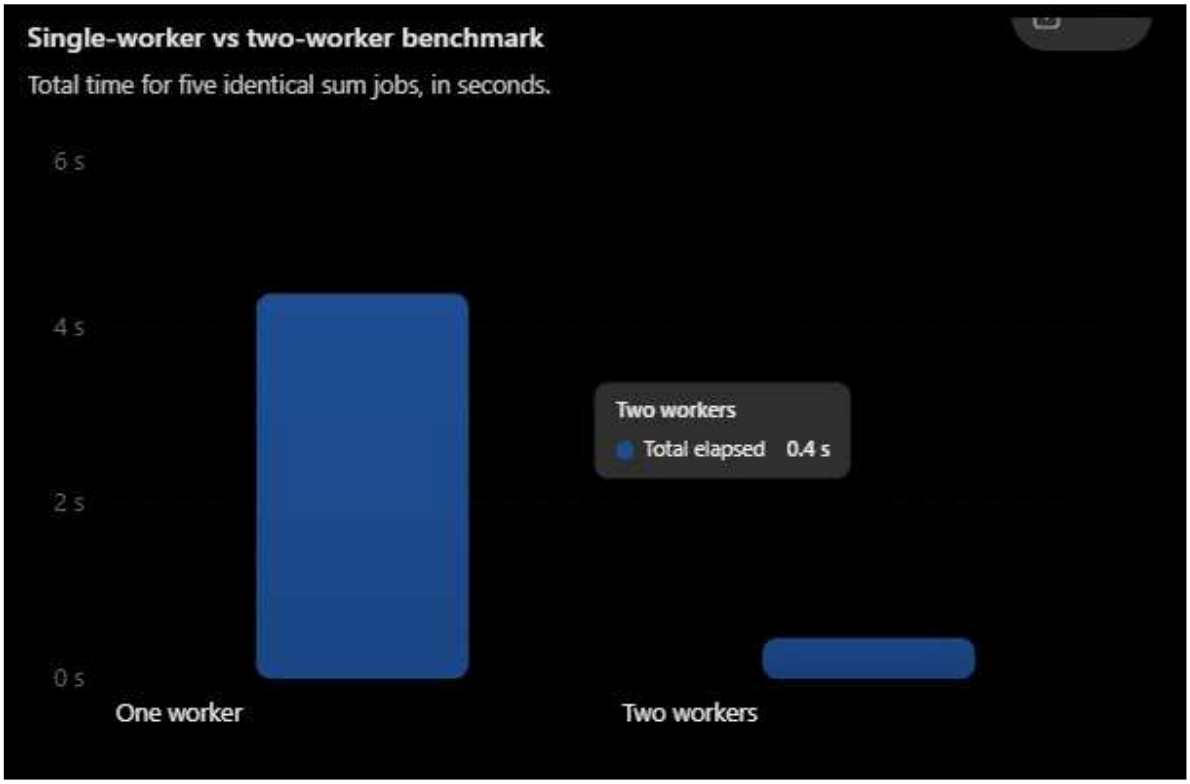
The process starts when the client sends a job request. The controller checks the request and, if it is valid, creates a unique job ID and adds the job to the queue. The scheduler then chooses an available worker based on its recorded load. Once the worker finishes the task, the controller saves the result and updates the job status so that the client can check it.



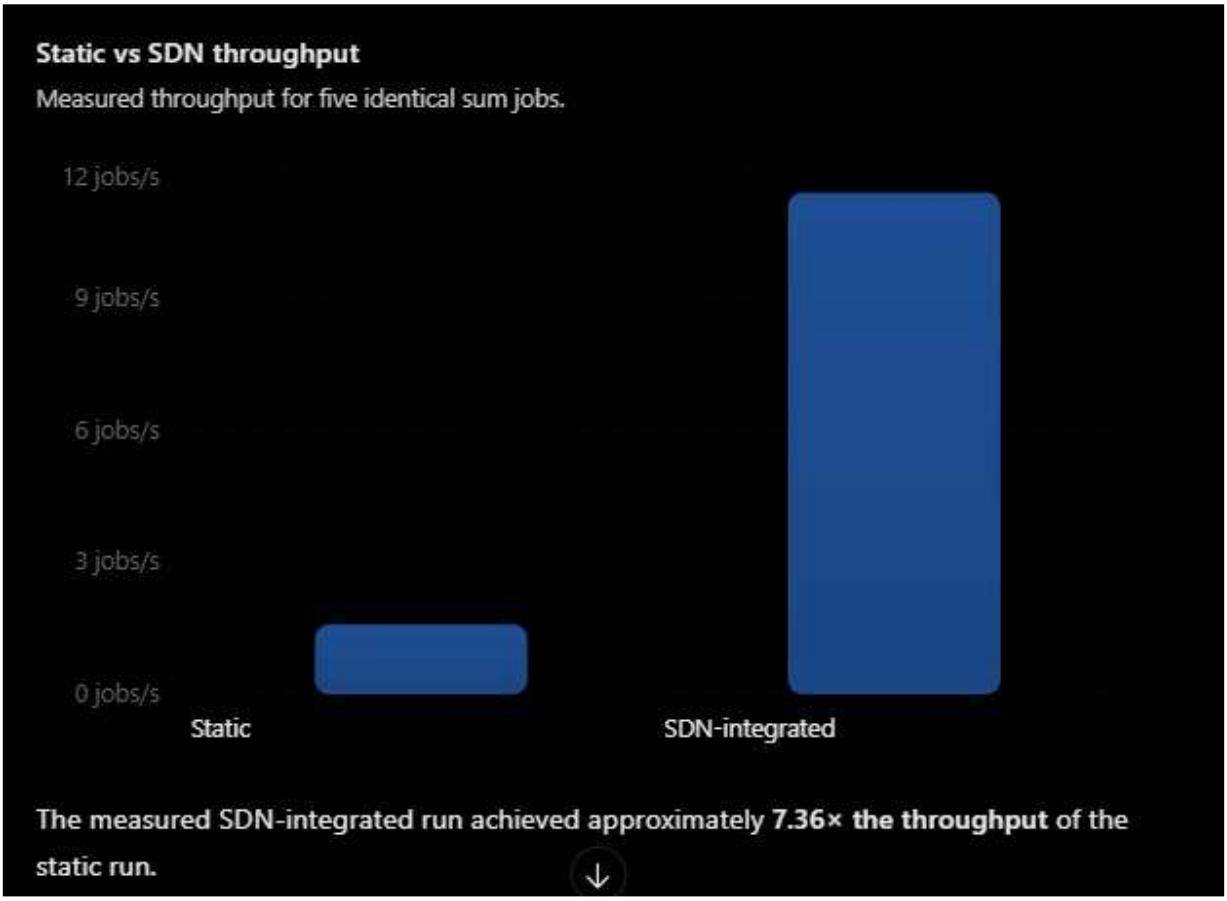
3 Worker / Network Failure and Recovery Flowchart

This flowchart shows how we plan to handle a worker or network failure. If a worker or network path becomes unavailable, the system should detect the problem. If the job has not finished, it can be placed back in the queue and assigned to another available worker. Ryu and OpenFlow can then be used to direct traffic through an alternative path. If the job has already completed, its result should be kept so that the same job is not unnecessarily executed again. This is the planned recovery process; it should only be described as fully implemented after we have tested it successfully.





Using two workers provides faster job execution compared to using a single worker because the workload is distributed among multiple workers.



Integrating Software Defined Networking improves the overall job-processing throughput and demonstrates the benefits of centralized network control and intelligent network management